

2026 Spring Graduation Project Interim Report

Knowledge Graph-Based Clinical Inference System

지식 그래프 기반 임상 추론 시스템



Team CHEERS

Project No.: 05 Division: A – Software / Artificial Intelligence

지도교수 (Advisor): 송길태 (GILTAE SONG)

BYAMBASUREN TUVSHINJARGAL (202355673) tuvshinjargal@pusan.ac.kr
GALBADRAKH BUYANDELGER (202255630) buyandelger@pusan.ac.kr
BAVUUJAV DELGERBAYAR (202355622) bayar22@pusan.ac.kr

Abstract

This interim report presents the progress of a knowledge graph-based clinical inference system that integrates clinical features, diseases, treatment drugs, contraindications, and drug–drug interactions into a unified multi-relational graph. Since the commencement report, the team has completed the main data preprocessing and graph construction stage. Four triple files were normalized into a common head–relation–tail structure and merged into a reference integrated graph containing approximately 1.53 million triples, 22,291 unique entities, and four relation types. Duplicate removal, reverse-pair handling, namespace normalization, type validation, and train/validation/test splitting were completed.

Three knowledge graph embedding models were assigned and implemented independently: TransE by Byambasuren Tuvshinjargal, ComplEx by Galbadrakh Buyandelger, and RotatE by Bavuujav Delgerbayar. The completed 64-dimensional TransE model achieved an overall filtered MRR of 0.174432 and Hits@10 of 0.346245 on the canonical project split, with typed relation-specific evaluation. The archived ComplEx run achieved an overall filtered MRR of 0.373992 and Hits@10 of 0.623632, but artifact inspection showed different entity mappings and train/test sizes; therefore, the two results are not treated as a fair head-to-head comparison. The RotatE experiment also produced model-specific filtered evaluation metrics. However, because its dataset snapshot and evaluation configuration were not aligned with the TransE reference experiment, the results are reported as independent preliminary evidence rather than as a direct comparison. The remaining work is to align all three models on the same canonical split and evaluator, select the final model, implement relation-specific inference, and integrate the inference engine with the planned FastAPI and React prototype.

Keywords: knowledge graph, clinical inference, knowledge graph embedding, TransE, ComplEx, RotatE, drug–drug interaction, PyKEEN

Contents

Abstract	i
1 Project Overview	1
1.1 Background and Motivation	1
1.2 Project Objectives	1
1.3 Interim Scope	2
2 Modifications to Requirements and Constraints	2
2.1 Initial Requirements	2
2.2 Modified Requirements	3
2.2.1 Addition of the contraindication relation	3
2.2.2 Expansion from two to three KGE models	3
2.2.3 Typed candidate restriction	3
2.2.4 Resource-aware experimental procedure	4
2.2.5 Staged application integration	4
2.3 Constraints and Countermeasures	4
3 Detailed Design and Changes	5
3.1 Updated Overall Architecture	5
3.2 Data Sources and DrugBank Access Limitation	5
3.2.1 DrugBank Access Limitation	6
3.3 UI/UX Prototype Design in Figma	7
3.4 Data Processing Pipeline	8
3.5 Reference Integrated Dataset Statistics	9
3.6 Model Design	9
3.6.1 TransE	10
3.6.2 ComplEx	10
3.6.3 RotatE	10
3.7 Typed Filtered Evaluation	10
3.8 Planned Inference Flow	10
4 Updated Project Plan	11
4.1 Progress Against the Original Plan	11
4.2 Revised Schedule	12
5 Team Member Progress	12
5.1 Role Assignment	13
5.2 Byambasuren Tuvshinjargal – Symptom-to-Disease Data and TransE	13
5.3 Galbadrakh Buyandelger – Disease-to-Drug Data and ComplEx	14
5.4 Bavuujav Delgerbayar – Drug–Drug Interaction Data and RotatE	14
5.5 Shared Team Work	15
6 Work Completed and Interim Results	15
6.1 Reference Knowledge Graph Construction Result	15
6.2 TransE Reference 64-Dimensional Experiment	15
6.2.1 Configuration	16
6.2.2 Overall Results	16

6.2.3	Typed Clinical-Relation Results	16
6.3	ComplEx Archived Experiment	17
6.3.1	Configuration and Artifact Audit	17
6.3.2	Overall Results	17
6.4	RotatE Experiment	17
6.4.1	Configuration and Artifact Audit	17
6.4.2	Overall Results	17
6.5	Summary of Preliminary Model Experiments	18
6.6	Engineering Improvements Completed	18
6.7	Interim Limitations	19
7	Planned Final Evaluation and Deliverables	19
7.1	Standardized Evaluation Protocol	19
7.2	Final Software Deliverables	20
7.3	Planned API Output	20
8	Conclusion	20

List of Tables

1	Interim project scope and status	2
2	Role of each embedding model in the comparison	3
3	Valid prediction candidates by relation	3
4	Current constraints and countermeasures	4
5	Relation distribution in the reference integrated knowledge graph	9
6	Updated phase status	11
7	Current team role assignment	13
8	Documented TransE reference configuration	16
9	TransE reference filtered evaluation results	16
10	TransE typed tail-prediction results by relation	16
11	Verified archived ComplEx overall filtered results	17
12	Overall Link Prediction Performance (Realistic Evaluation)	17
13	Status of the independent preliminary model experiments	18

List of Figures

1	Updated architecture and interim completion boundary	5
2	DrugBank notification stating that academic dataset downloads were temporarily paused	6
3	Figma Version 1 UI/UX prototype for the clinical inference system	8
4	Reference knowledge graph construction pipeline	9
5	Typed filtered evaluation for clinically valid rankings	10

1. Project Overview

1.1 Background and Motivation

Biomedical knowledge is inherently relational. A clinical feature may be related to several diseases, a disease may be treated by multiple drugs, and each candidate drug may interact with other medications or be contraindicated for a particular condition. These relationships are commonly stored in separate datasets, which makes integrated multi-step reasoning difficult.

This project represents the relationships as a unified knowledge graph and applies knowledge graph embedding (KGE) models to link prediction. The target reasoning path is:

Clinical feature $\rightarrow$ disease $\rightarrow$ treatment drug $\rightarrow$ interaction and contraindication check.

The planned final system will receive one or more user-provided clinical features, rank candidate diseases, rank candidate treatment drugs, and display medication safety warnings. The system is an academic research prototype and is not intended to replace professional diagnosis or treatment.

1.2 Project Objectives

The objectives of this project are to design, implement, and evaluate a knowledge graph-based clinical inference system capable of integrating heterogeneous biomedical knowledge into a unified reasoning framework. Specifically, the project aims to:

1. Construct a unified biomedical knowledge graph by integrating heterogeneous data sources containing clinical feature–disease, disease–drug, contraindication, and drug–drug interaction relationships.
2. Design a standardized data engineering pipeline for triple generation, entity normalization, namespace management, duplicate removal, and canonical reference dataset construction.
3. Develop, implement, and evaluate multiple knowledge graph embedding models (TransE, ComplEx, and RotatE) for biomedical link prediction and analyze their suitability for clinical inference under a standardized evaluation protocol.
4. Implement type-aware clinical inference that predicts candidate diseases from clinical features, recommends treatment drugs, detects contraindications, and identifies potential drug–drug interactions while preserving semantic consistency between entity types.
5. Design a scalable clinical inference architecture consisting of a FastAPI backend, a React-based web interface, and an interactive knowledge graph visualization that presents prediction results in an intuitive and explainable manner.
6. Validate the proposed system through reproducible experiments using filtered link-prediction metrics, typed relation-specific evaluation, and comprehensive documentation of experimental configurations, limitations, and future improvements.

1.3 Interim Scope

The primary completed work is data engineering and model experimentation. Application integration remains a next-phase task. The current scope is therefore divided as follows:

Table 1: Interim project scope and status

Area	Work item	Status
Data engineering	Source review, triple conversion, normalization, duplicate removal, type validation, Reference integrated graph construction	Completed
Dataset preparation	Reference dataset construction, canonical entity/relation mappings, and train/validation/test split	Completed
TransE	Reference 64-dimensional experiment, early stopping, filtered and typed evaluation, saved artifacts	Completed
ComplEx	64d archived run, standard filtered evaluation, artifact audit	Completed / requires canonical rerun
RotatE	Model implementation, training, and evaluation run	Completed / verifying
Model comparison	Shared evaluation protocol and canonical comparison setup	In progress
Inference module	Relation-specific top-K prediction and safety checks	Planned
Web prototype	FastAPI, React, and graph visualization	Planned

2. Modifications to Requirements and Constraints

2.1 Initial Requirements

The commencement report defined the following requirements:

- 1) Collect clinical feature–disease, disease–drug, and drug–drug interaction relationships.
- 2) Convert the data into a unified knowledge graph.
- 3) Train KGE models, initially focusing on TransE and RotatE.
- 4) Infer probable diseases from clinical features.
- 5) Recommend candidate treatment drugs for inferred diseases.
- 6) Detect drug–drug interactions with the user’s current medication.
- 7) Expose inference through FastAPI and visualize results in React.
- 8) Evaluate the model with Hits@K and Mean Reciprocal Rank.

2.2 Modified Requirements

2.2.1 Addition of the contraindication relation

The initial design emphasized three relation families. During data integration, a fourth relation, **contraindicated_for**, was retained because it provides negative safety information that cannot be represented by **treated_by** or **interacts_with**. The final graph therefore uses four relations:

- **related_to**: clinical feature $\rightarrow$ disease
- **treated_by**: disease $\rightarrow$ drug
- **contraindicated_for**: disease or condition $\rightarrow$ drug, according to the normalized stored direction
- **interacts_with**: drug $\leftrightarrow$ drug

2.2.2 Expansion from two to three KGE models

The original plan focused on TransE and RotatE. ComplEx was added because the graph contains a dominant symmetric drug–drug interaction relation. The three-model comparison now has the following purpose:

Table 2: Role of each embedding model in the comparison

Model	Main property	Reason for inclusion
TransE	Translation in real vector space	Simple and efficient baseline; suitable for hierarchical relations
ComplEx	Complex-valued bilinear scoring	Represents symmetric and antisymmetric patterns
RotatE	Rotation in complex vector space	Represents symmetry, antisymmetry, inversion, and composition

2.2.3 Typed candidate restriction

A general KGE evaluator considers every entity as a possible answer. In a heterogeneous biomedical graph, this may rank a drug as the tail of a clinical feature–disease query or a disease as the tail of a DDI query. The evaluation and inference requirements were therefore revised to include type-aware candidate restriction.

Table 3: Valid prediction candidates by relation

Relation	Expected head type	Expected tail type
related_to	Clinical feature	Disease
treated_by	Disease	Drug
contraindicated_for	Disease / condition	Drug
interacts_with	Drug	Drug

2.2.4 Resource-aware experimental procedure

The completed graph contains more than 1.53 million triples. Full training with large dimensions and batches caused memory limitations or long execution times. The experimental procedure was changed to:

- 1) Validate the data and mappings.
- 2) Run a short smoke test.
- 3) Run a small CPU or reduced-memory pilot.
- 4) Save checkpoints and best model weights.
- 5) Move to a GPU environment for larger controlled experiments.
- 6) Evaluate the three models under the same final protocol.

2.2.5 Staged application integration

The backend and frontend will be integrated only after entity mappings, output schemas, and final model selection are stabilized. This prevents repeated changes to the API while model outputs are still changing.

2.3 Constraints and Countermeasures

Table 4: Current constraints and countermeasures

Constraint	Observed issue	Countermeasure
Relation imbalance	<code>interacts_with</code> accounts for approximately 87.51% of all triples, while <code>treated_by</code> accounts for approximately 0.61%. Overall metrics may therefore be dominated by DDI prediction.	Report relation-specific metrics; consider balanced sampling, relation-weighted analysis, and separate evaluation for the clinically important relations.
Memory limitations	High embedding dimensions and batch sizes can exceed available RAM or GPU memory.	Reduce embedding dimension and batch size; use checkpointing; test configurations before full training; use Colab or another GPU environment for the final controlled run.
Long evaluation time	Filtered ranking over 22,291 entities and 152,252 test triples is computationally expensive.	Use typed candidate sets; evaluate a representative subset during debugging; run the complete evaluation only for the final selected configurations.
Entity inconsistency	The same concept may use different identifiers or names across source files.	Use type-specific namespaces, normalize identifiers, remove duplicates, and validate relation endpoint types.

Constraint	Observed issue	Countermeasure
Metric comparability	Current TransE and ComplEx artifacts use different train/test counts, entity mappings, batch sizes, model-selection procedures, and evaluator coverage.	Do not make a final model claim from the current raw numbers; retrain and evaluate all models using one canonical protocol.
Clinical reliability	Link prediction scores indicate graph plausibility, not verified medical correctness.	Display outputs as research predictions, include graph evidence and confidence/rank information, and show a clear non-diagnostic disclaimer.

3. Detailed Design and Changes

3.1 Updated Overall Architecture

The architecture remains a three-tier system, but the current report separates the completed data and model-development layers from the application layer. Although backend and frontend implementation remains planned, the first UI/UX prototype has already been designed in Figma to define the expected user flow, screen structure, result presentation, and graph-visualization requirements.

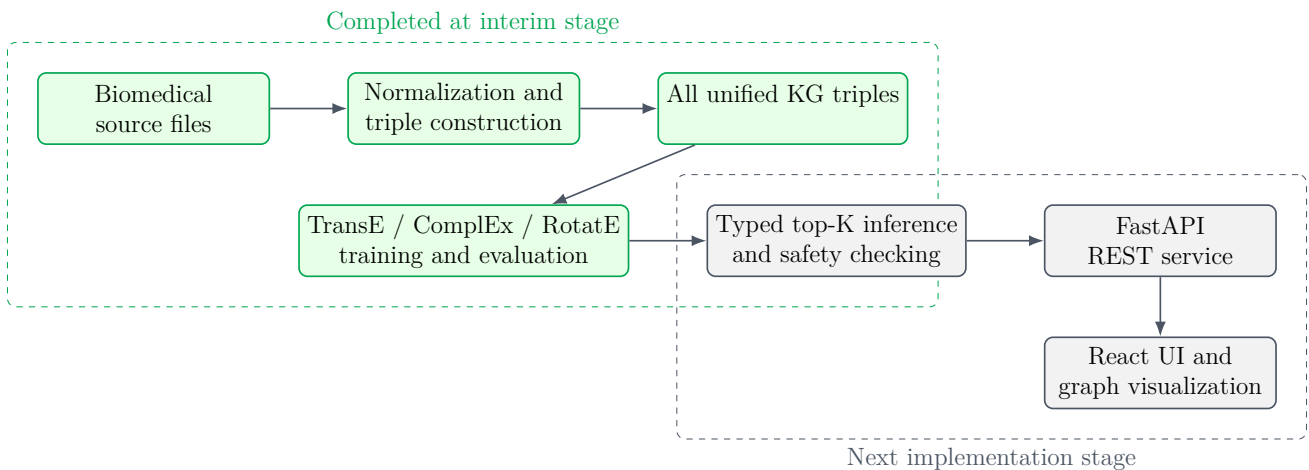


Figure 1: Updated architecture and interim completion boundary

3.2 Data Sources and DrugBank Access Limitation

The project used multiple publicly accessible biomedical data sources because no single dataset contained all of the clinical feature–disease, disease–drug, contraindication, and drug–drug interaction relationships required for the proposed inference pipeline.

The data sources used during the interim stage include:

- **PrimeKG:** PrimeKG was used as a major source of structured biomedical relationships,

including clinical-feature-disease associations, drug-disease relationships, treatment indications, and contraindication information.

- **DDInter:** DDInter was used as a major source of drug-drug interaction records. The collected records were converted into standardized `interacts_with` triples after drug-name normalization, duplicate removal, and reverse-pair processing.
- **Disease-Symptom Dataset (Kaggle):** A publicly available Disease-Symptom Dataset was used as a supplementary source for symptom-disease relationships. The dataset contains 773 disease classes and approximately 246,945 records. Symptom features were converted into standardized `related_to` triples after preprocessing, normalization, and duplicate removal.

The Kaggle dataset was not assumed to share a common schema or identifier system with the other sources. Therefore, its columns, entity names, relation directions, and duplicate records were reviewed before integration. Only records that could be mapped to the project’s clinical feature, disease, drug, and relation types were retained.

3.2.1 DrugBank Access Limitation

DrugBank was originally selected as an additional source of curated disease-drug treatment information. The team submitted an academic access request; however, DrugBank informed the applicant that all DrugBank Academic dataset downloads were temporarily paused while its distribution process was being updated.

Consequently, the DrugBank dataset was not obtained and was not included in the current reference integrated dataset or preliminary model experiments.

The current disease-to-drug pipeline therefore uses treatment-indication and drug-disease relationships obtained from the accessible biomedical source files, including PrimeKG, rather than from DrugBank.

This is treated as an external data-access constraint rather than a change to the overall inference objective. DrugBank integration may be reconsidered if academic downloads resume and if the licensing requirements and remaining project schedule permit its inclusion.

 **Important: All Academic downloads are temporarily paused.**

All Academic DrugBank dataset downloads are temporarily paused while we update how we distribute DrugBank data.

We know these datasets matter to your work, and we’ll restore access as soon as the updated program is in place.

Want to be notified when downloads resume? [Sign up here](#).

Thank you for your patience and continued support of DrugBank.

Figure 2: DrugBank notification stating that academic dataset downloads were temporarily paused

3.3 UI/UX Prototype Design in Figma

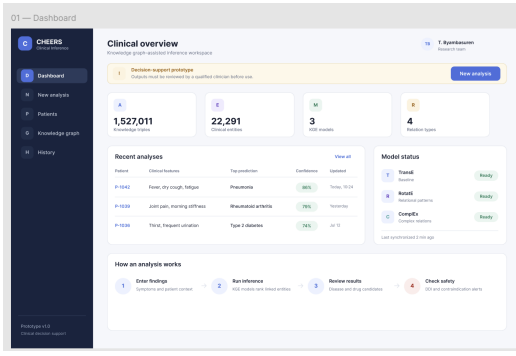
A first-version UI/UX prototype was created in Figma before frontend implementation.

The prototype translates the planned clinical inference process into a user-facing workflow and establishes a shared visual specification for the future React implementation.

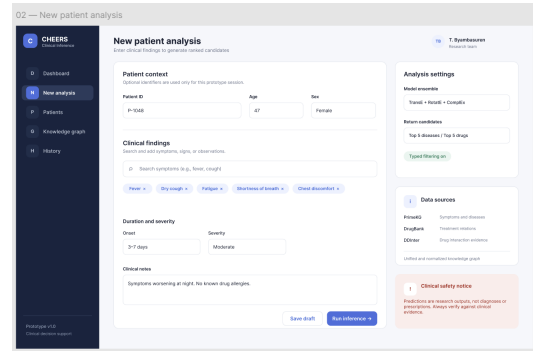
The Version 1 prototype defines the principal user flow and presentation requirements for the planned web application, including:

- a landing or introduction screen explaining the purpose of the research prototype;
- a clinical-feature input interface;
- an optional current-medication input field;
- ranked disease prediction results;
- candidate treatment-drug results;
- contraindication and DDI warning components;
- a knowledge graph visualization area;
- loading, empty, and error-state requirements; and
- a visible disclaimer stating that the outputs are research predictions rather than medical diagnoses or prescriptions.

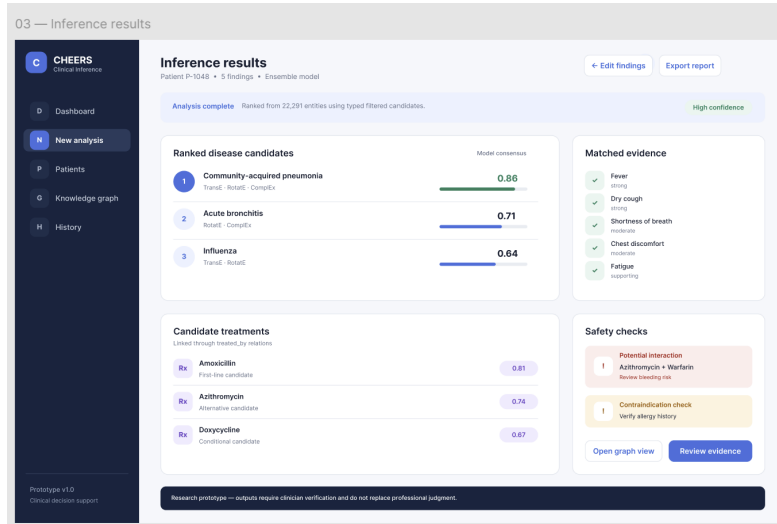
The prototype is used as the design baseline for the planned React frontend. The terminology, screen flow, component hierarchy, and warning presentation will be revised after advisor feedback and FastAPI response-schema confirmation.



(a) Figma Version 1 screen



(b) Clinical inference interface



(c) Clinical inference results

Figure 3: Figma Version 1 UI/UX prototype for the clinical inference system

3.4 Data Processing Pipeline

Four relation-specific source files were processed and converted into a common triple structure. The preprocessing pipeline is summarized in Figure 4.

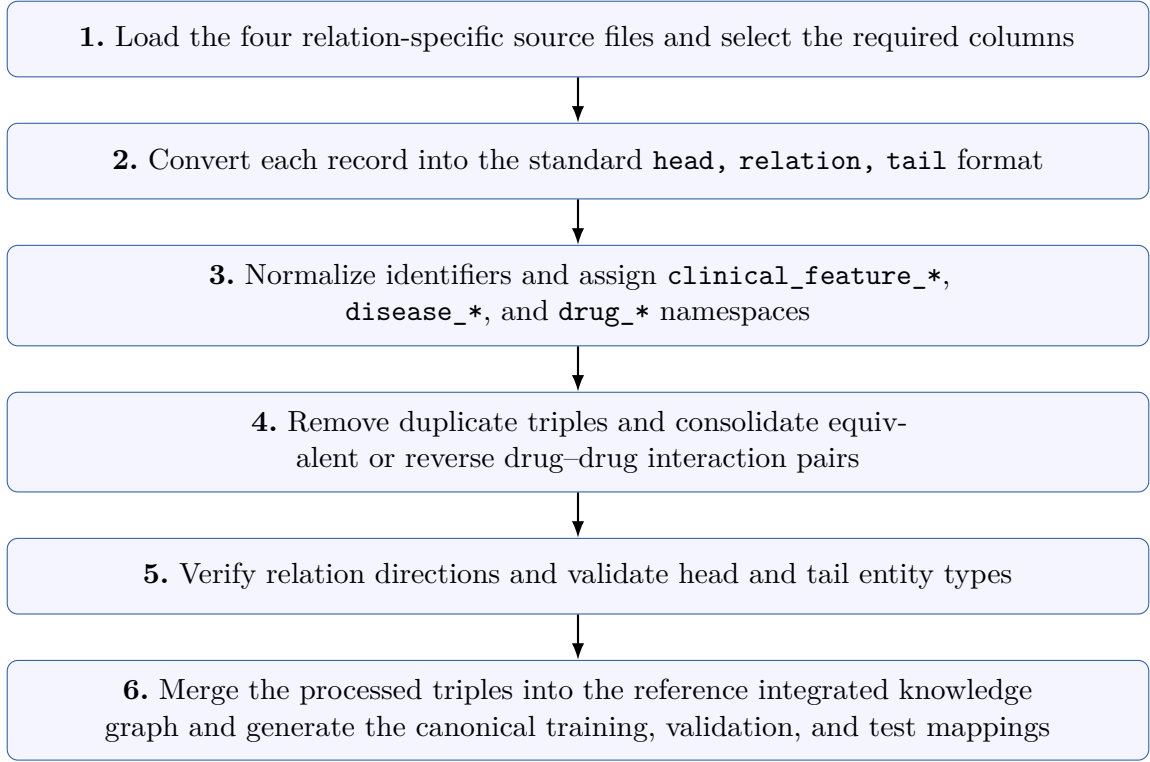


Figure 4: Reference knowledge graph construction pipeline

3.5 Reference Integrated Dataset Statistics

Table 5: Relation distribution in the reference integrated knowledge graph

Relation	Triples	Share	Primary purpose
related_to	150,634	9.86%	Clinical feature–disease association
treated_by	9,388	0.61%	Disease–treatment drug relation
contraindicated_for	30,675	2.01%	Contraindication information
interacts_with	1,336,314	87.51%	Drug–drug interaction
Total	1,527,011	100.00%	

The graph contains **22,291 unique entities**. The stored namespace and relation patterns are:

1	clinical_feature_*	-> related_to	-> disease_*
2	disease_*	-> treated_by	-> drug_*
3	disease_*	-> contraindicated_for	-> drug_*
4	drug_*	<-> interacts_with	<-> drug_*

Listing 1: Unified triple patterns

3.6 Model Design

3.6.1 TransE

TransE models a valid triple (h, r, t) as a translation in real-valued embedding space:

$$\mathbf{h} + \mathbf{r} \approx \mathbf{t}.$$

A common distance-based score is:

$$f_r(h, t) = -\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_p$$

It is used as the computationally efficient baseline model.

3.6.2 ComplEx

ComplEx uses complex-valued embeddings and the score:

$$f_r(h, t) = \text{Re}(\langle \mathbf{h}, \mathbf{r}, \bar{\mathbf{t}} \rangle).$$

Its ability to represent both symmetric and antisymmetric patterns is relevant to the mixed relation structure of the graph.

3.6.3 RotatE

RotatE models each relation as a rotation in complex space:

$$f(h, r, t) = -\|\mathbf{h} \circ \mathbf{r} - \mathbf{t}\|$$

This model can represent symmetry, antisymmetry, inversion, and composition, making it suitable for testing the graph's relation patterns.

3.7 Typed Filtered Evaluation

For a test triple (h, r, t) , filtered evaluation removes other known true triples from the candidate list before computing the rank of t . The revised evaluator additionally restricts the candidate set by relation-specific entity type.

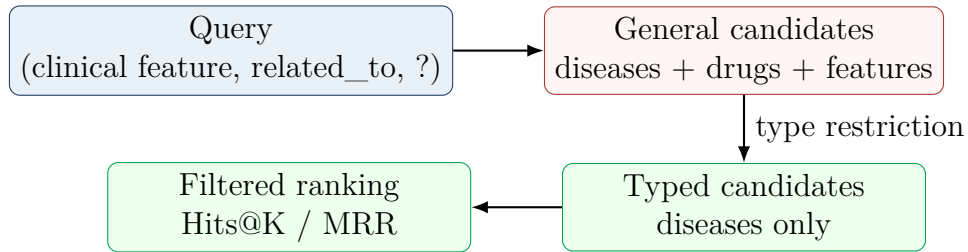


Figure 5: Typed filtered evaluation for clinically valid rankings

3.8 Planned Inference Flow

After final model selection, the inference module will execute the following sequence:

- 1) Receive one or more clinical features.
- 2) Rank disease candidates using `related_to`.

- 3) Rank candidate drugs using `treated_by` for the selected disease candidates.
- 4) Remove or flag drugs connected by `contraindicated_for`.
- 5) Compare candidate drugs with current medications using `interacts_with`.
- 6) Return ranked predictions, graph edges, and warning metadata as JSON.

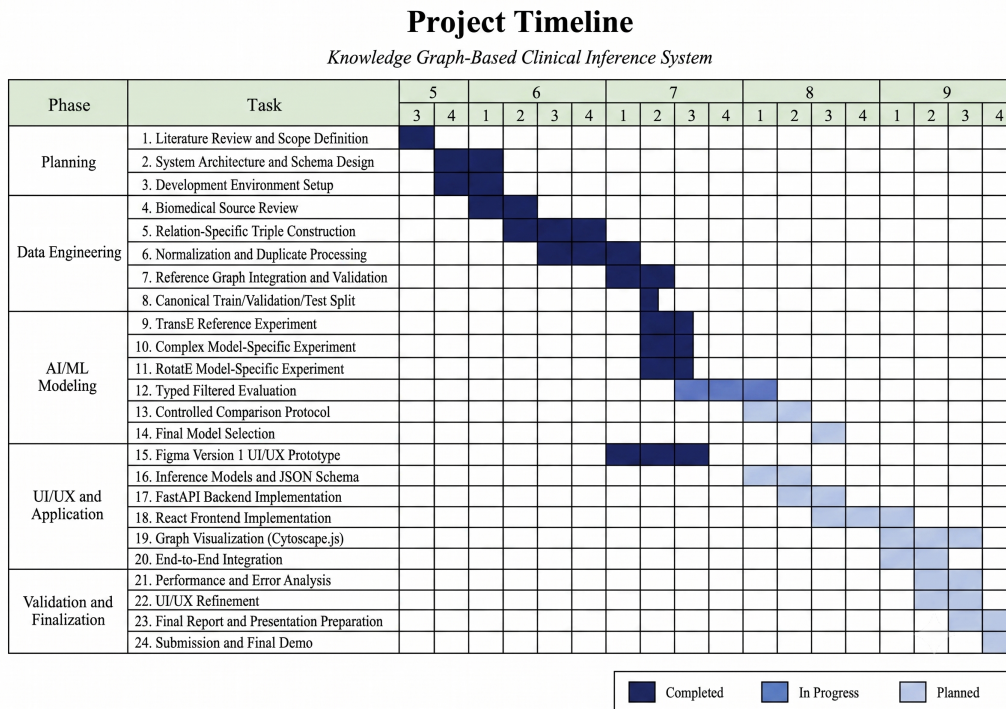
4. Updated Project Plan

4.1 Progress Against the Original Plan

Table 6: Updated phase status

Phase	Work	Deliverable	Status
1	Source analysis and schema design	Relation and entity-type definition	Completed
2	Triple conversion and normalization	Four cleaned triple files	Completed
3	Reference integrated graph construction	Approximately 1.53 million triple graph	Completed
4	Dataset splitting and mappings	Train/validation/test files	Completed
5	TransE experiment	Reference 64-dimensional experiment, metrics, typed evaluation, and saved weights	Completed
6	ComplEx experiment	Archived 64d model and standard filtered metrics	Completed / rerun needed
7	RotatE experiment	RotatE training/evaluation run	Completed
8	Standardized comparison	One comparable result table	In progress
9	Inference engine	Typed disease/drug/DDI predictions	Planned
10	Web integration	FastAPI + React prototype	Planned
11	Final evaluation	End-to-end test and error analysis	Planned

4.2 Revised Schedule



Remaining Work

- 1) Standardize the evaluation protocol.
- 2) Retrain all models on the canonical dataset.
- 3) Compare model performance.
- 4) Select the final model.
- 5) Implement top-K inference.
- 6) Integrate FastAPI and React.
- 7) Perform end-to-end testing.

5. Team Member Progress

5.1 Role Assignment

Table 7: Current team role assignment

Name	Student ID	Assigned data/model	Main responsibilities
Byambasuren Tuvshinjargal	202355673	Symptom–disease data / TransE	Construction and validation of related_to triples; shared KG integration and split generation; TransE training, typed filtered evaluation, metric analysis, and symptom-to-disease inference preparation
Galbadrakh Buyandelger	202255630	Disease–drug data / ComplEx	Construction and validation of treated_by triples; shared KG verification; ComplEx training and optimization, filtered evaluation, metric analysis, and disease-to-drug recommendation preparation
Bavuujav Delgerbayar	202355622	Drug–drug interaction data / RotatE	Construction and validation of interacts_with triples; duplicate and reverse-pair handling; RotatE training and evaluation, metric analysis, and DDI safety-checking preparation

5.2 Byambasuren Tuvshinjargal – Symptom-to-Disease Data and TransE

Completed work:

- Collected and processed symptom-to-disease data and converted it into standardized **related_to** triples.
- Performed entity normalization, identifier assignment, duplicate removal, relation-direction verification, and entity-type validation.
- Contributed to integrating all relation files into the reference integrated knowledge graph and generating consistent training, validation, and test datasets.
- Implemented, trained, and evaluated TransE using smoke tests, checkpointing, standard and typed filtered evaluation, MRR, Mean Rank, and Hits@K.
- Contributed to model comparison, result interpretation, technical documentation, and interim report preparation.
- Designed the first Figma UI/UX prototype for the clinical inference web application, defining the overall workflow, screen layout, and visualization concept.

Planned work:

- Maintain TransE as the reproducible baseline and complete standardized comparison under the shared protocol.
- Implement and validate the symptom-to-disease top-K inference function for backend integration.

5.3 Galbadrakh Buyandelger – Disease-to-Drug Data and ComplEx

Completed work:

- Collected and processed disease-to-drug treatment data and converted it into standardized `treated_by` triples.
- Performed entity normalization, identifier assignment, duplicate removal, relation-direction verification, and entity-type validation.
- Contributed to unified knowledge graph integration and verification of the shared dataset and split consistency.
- Implemented, trained, optimized, and evaluated ComplEx using reduced-memory hyperparameters, checkpointing, filtered evaluation, MRR, Mean Rank, and Hits@K.
- Contributed to model comparison, result interpretation, technical documentation, and interim report preparation.

Planned work:

- Rerun ComplEx on the canonical shared split with the same validation and typed filtered evaluation protocol.
- Implement and validate the disease-to-drug recommendation function for the inference pipeline.

5.4 Bavuujav Delgerbayar – Drug–Drug Interaction Data and RotatE

Completed work:

- Collected and processed drug–drug interaction data and converted it into standardized `interacts_with` triples.
- Performed drug identifier normalization, duplicate removal, reverse-pair consolidation, relation verification, and entity-type validation.
- Contributed to unified knowledge graph integration and verification of the quality and consistency of the final dataset.
- Implemented, trained, optimized, and evaluated RotatE using configured hyperparameters, checkpointing, filtered evaluation, MRR, Mean Rank, and Hits@K.
- Contributed to model comparison, result interpretation, technical documentation, and interim report preparation.

Planned work:

- Confirm and archive the exact RotatE configuration and final evaluation metrics under the shared protocol.
- Implement and validate the drug–drug interaction safety-checking function for backend integration.

5.5 Shared Team Work

The team jointly reviews data validity, model outputs, documentation, experimental comparability, and the future API response format. The next shared milestone is a single standardized table comparing all three models.

6. Work Completed and Interim Results

6.1 Reference Knowledge Graph Construction Result

The completed reference integrated graph is the main interim deliverable. It contains:

- 1,527,011 triples,
- 22,291 unique entities,
- four relation types,
- canonical entity and relation mappings within the reference dataset,
- train/validation/test splits,
- normalized entity namespaces,
- duplicate removal and endpoint-type validation.

The reference graph serves as the canonical dataset for subsequent controlled training and evaluation.

6.2 TransE Reference 64-Dimensional Experiment

6.2.1 Configuration

Table 8: Documented TransE reference configuration

Parameter	Value
Model	TransE
Dataset	Canonical project split
Training / validation / test triples	1,222,485 / 152,274 / 152,252
Entities / relations	22,291 / 4
Embedding dimension	64
Maximum epochs	50
Completed / selected epoch	Stopped at epoch 40; best checkpoint at epoch 20
Optimizer / learning rate	Adam / 0.001
Loss	MarginRankingLoss
Training batch size	1,024
Random seed	42
Evaluation	Standard filtered and typed relation-specific evaluation
Recorded device	CPU
Training / evaluation time	3h 02m 51s / 0h 43m 38s

6.2.2 Overall Results

Table 9: TransE reference filtered evaluation results

Metric	Result
MRR	0.174432
Hits@1	0.088547
Hits@3	0.188424
Hits@5	0.248923
Hits@10	0.346245
Mean Rank	188.69

The TransE package is the current reproducible reference experiment because its canonical split, mappings, checkpoint selection, standard metrics, typed relation-specific metrics, and saved model artifacts are documented.

6.2.3 Typed Clinical-Relation Results

Table 10: TransE typed tail-prediction results by relation

Relation	MRR	H@1	H@3	H@5	H@10	MR
related_to	0.022515	0.009060	0.018665	0.024591	0.042098	1143.26
treated_by	0.075499	0.027655	0.087389	0.111726	0.158186	467.67
contraindicated_for	0.049079	0.015748	0.041010	0.064961	0.104987	260.98
interacts_with	0.178153	0.090488	0.191498	0.254124	0.354026	88.13

TransE is strongest on `interacts_with` and weakest on `related_to`. This result reflects both relation difficulty and the severe graph imbalance toward DDI triples.

6.3 ComplEx Archived Experiment

6.3.1 Configuration and Artifact Audit

The archived ComplEx run used a 64-dimensional embedding and batch size 128 for 30 epochs. Its saved model and mappings are usable with the matching PyKEEN environment.

6.3.2 Overall Results

Table 11: Verified archived ComplEx overall filtered results

Metric	Result
MRR	0.373992
Hits@1	0.255167
Hits@3	0.419537
Hits@5	0.504078
Hits@10	0.623632
Mean Rank	278.86

The archived ComplEx experiment demonstrates successful implementation and evaluation under its own configuration. Because its dataset snapshot and mappings differ from those of the reference TransE experiment, these metrics are not interpreted as a direct performance comparison.

6.4 RotatE Experiment

6.4.1 Configuration and Artifact Audit

The RotatE knowledge graph embedding model was trained on a merged clinical knowledge graph containing 1.52 million triples and four relation types. Training was conducted for 20 epochs using the Adam optimizer with a learning rate of 0.001 on a CPU-only environment. The model demonstrated stable convergence, with the training loss decreasing from 0.2854 to 0.0381.

6.4.2 Overall Results

Table 12: Overall Link Prediction Performance (Realistic Evaluation)

Metric	Result
Mean Reciprocal Rank (MRR)	0.319086
Mean Rank (MR)	126.28
Hits@1	0.227394
Hits@3	0.348504
Hits@5	0.409554
Hits@10	0.502721

The RotatE model achieved a Mean Rank (MR) of 126.28, indicating that the correct entity was ranked, on average, 126th among all candidate entities. Although Mean Rank provides an overall

measure of ranking performance, it is sensitive to outlier predictions. Therefore, Mean Reciprocal Rank (MRR) and Hits@K are considered more reliable evaluation metrics. The model achieved an MRR of 0.3191 and a Hits@10 score of 50.27 indicating that the correct entity was ranked within the top ten predictions in approximately half of all link prediction tasks.

6.5 Summary of Preliminary Model Experiments

Table 13: Status of the independent preliminary model experiments

Model	Experiment status	Evidence obtained	Directly comparable?
TransE	Reference experiment completed	Documented canonical split, checkpoint selection, standard filtered metrics, and typed relation-specific metrics	No
ComplEx	Archived model-specific experiment completed	Saved model, mappings, configuration, and standard filtered metrics	No
RotatE	Archived model-specific experiment completed	Training convergence and standard filtered evaluation metrics	No

The three experiments demonstrate successful implementation of independent model pipelines. Because their dataset snapshots, mappings, and evaluation conditions differ, the current results do not establish that one model outperforms another.

Important interpretation rule. The current rows must not be used as a direct ranking of the three models because the configurations are not yet identical. The final report will compare models only after using the same split, seed, evaluator, negative sampling policy, training budget, and reporting procedure.

6.6 Engineering Improvements Completed

The following reliability improvements were introduced during the interim stage:

- 1) Short smoke tests before long training runs.
- 2) Consistent entity and relation mappings across the canonical reference training, validation, and test splits.
- 3) Checkpoint saving and resume support.
- 4) Best-model-weight storage.
- 5) Typed candidate filtering for clinically valid evaluation.
- 6) Separate output directories for model configurations.
- 7) GPU-ready bundle preparation without unnecessary raw data.

- 8) Explicit documentation of metrics that are confirmed versus metrics still awaiting verification.

6.7 Interim Limitations

- The relation distribution is highly imbalanced toward drug–drug interactions.
- The current TransE and ComplEx runs use different verified data splits and entity mappings.
- The ComplEx archive lacks equivalent typed per-relation metrics and validation split evidence.
- The archived RotatE experiment still requires complete documentation of its dataset snapshot and training configuration.
- The three current model runs are not yet directly comparable.
- The inference API and web interface have not yet been completed.
- Clinical validity must be checked separately from structural link-prediction performance.

7. Planned Final Evaluation and Deliverables

7.1 Standardized Evaluation Protocol

The final comparison will record the following for every model:

- identical train/validation/test triples,
- identical entity and relation mappings,
- random seed,
- embedding dimension,
- training epochs or early-stopping policy,
- batch size and negative sampling configuration,
- optimizer and learning rate,
- filtered and typed filtered metrics,
- overall and relation-specific Hits@1, Hits@3, Hits@5, Hits@10, MRR, and Mean Rank,
- training time, evaluation time, and peak memory usage where available.

7.2 Final Software Deliverables

- 1) Cleaned unified triple dataset and split files.
- 2) Reproducible training scripts for TransE, ComplEx, and RotatE.
- 3) Saved final model and mappings.
- 4) Typed inference module.
- 5) FastAPI endpoint for clinical inference.
- 6) React user interface.
- 7) Interactive graph visualization of the reasoning path.
- 8) Final report, presentation, and demonstration materials.

7.3 Planned API Output

A future inference response will follow a structure similar to:

```

1 {
2   "input_features": ["clinical_feature_fever"],
3   "diseases": [
4     {"id": "disease_example", "score": 0.91, "rank": 1}
5   ],
6   "candidate_drugs": [
7     {
8       "id": "drug_example",
9       "score": 0.86,
10      "contraindicated": false,
11      "ddi_warnings": ["drug_current_medication"]
12    }
13  ],
14  "reasoning_edges": [
15    ["clinical_feature_fever", "related_to", "disease_example"],
16    ["disease_example", "treated_by", "drug_example"]
17  ],
18  "disclaimer": "Research prototype; not medical advice."
19 }
```

Listing 2: Planned inference response schema

8. Conclusion

At the interim stage, the team has completed the core data engineering work and established three independent KGE experiment pipelines. The reference integrated graph contains 1,527,011 triples and 22,291 entities across four clinical relation types. Tuvshinjargal completed the symptom-to-disease data pipeline and TransE baseline. Tuvshinjargal also designed the first Figma UI/UX prototype, providing the basis for the planned React frontend implementation. Buyandelger

completed the disease-to-drug data pipeline and ComplEx experiment, and Delgerbayar completed the drug–drug interaction data pipeline and RotatE experiment.

The preliminary results demonstrate that the graph can be trained and evaluated successfully, but they also reveal the need for a controlled comparison. The next phase will focus on archiving all experiment configurations, completing the standardized evaluation protocol, selecting the final model, and implementing the end-to-end inference prototype. The final system will emphasize transparent graph-based reasoning and medication safety warnings while clearly identifying itself as an academic research tool rather than a medical diagnostic service.

References

- [1] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko, “Translating embeddings for modeling multi-relational data,” *Advances in Neural Information Processing Systems*, vol. 26, 2013.
- [2] T. Trouillon, J. Welbl, S. Riedel, E. Gaussier, and G. Bouchard, “Complex embeddings for simple link prediction,” *Proceedings of the 33rd International Conference on Machine Learning*, pp. 2071–2080, 2016.
- [3] Z. Sun, Z. H. Deng, J. Y. Nie, and J. Tang, “RotatE: Knowledge graph embedding by relational rotation in complex space,” *International Conference on Learning Representations*, 2019.
- [4] P. Chandak, K. Huang, and M. Zitnik, “Building a knowledge graph to enable precision medicine,” *Scientific Data*, vol. 10, article 67, 2023.
- [5] M. Ali et al., “PyKEEN 1.0: A Python library for training and evaluating knowledge graph embeddings,” *Journal of Machine Learning Research*, vol. 22, no. 82, pp. 1–6, 2021.
- [6] D. S. Wishart et al., “DrugBank 5.0: A major update to the DrugBank database for 2018,” *Nucleic Acids Research*, vol. 46, no. D1, pp. D1074–D1082, 2018.
- [7] Y. H. Feng et al., “DDInter: An online drug–drug interaction database towards improving clinical decision-making and patient safety,” *Nucleic Acids Research*, vol. 50, no. D1, pp. D1200–D1207, 2022.
- [8] Dhivyesh RK, “Disease–Symptom Dataset,” *Kaggle*, Version 1. [Online]. Available: <https://www.kaggle.com/datasets/dhivyeshrk/diseases-and-symptoms-dataset>.